

Namespace Deodesy.Library

Classes

[Coordinate](#)

Represents a point on the Earth's surface using Latitude and Longitude.

[Dms](#)

Provides methods for handling Degrees, Minutes, Seconds (DMS) conversions and formatting.

[LatLonSpherical](#)

Provides functions for spherical geodesy, based on the Haversine formula and other spherical geometry calculations. These functions are adapted from Chris Veness's original JavaScript geodesy library.

Class Coordinate

Namespace: [Deodesy.Library](#)

Assembly: Deodesy.Library.dll

Represents a point on the Earth's surface using Latitude and Longitude.

```
public class Coordinate
```

Inheritance

[object](#)  ← Coordinate

Inherited Members

[object.Equals\(object, object\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) ,
[object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

Coordinate(double, double)

Initializes a new instance of the [Coordinate](#) class.

```
public Coordinate(double latitude = 0, double longitude = 0)
```

Parameters

latitude [double](#) 

The latitude in degrees (-90 to 90). Defaults to 0.

longitude [double](#) 

The longitude in degrees (-180 to 180). Defaults to 0.

Examples

```
var coord1 = new Coordinate(); // Latitude: 0, Longitude: 0  
var coord2 = new Coordinate(30, -120); // Latitude: 30, Longitude: -120
```

Properties

Latitude

Gets or sets the latitude in degrees. The value is wrapped to the range -90 to 90.

```
public double Latitude { get; set; }
```

Property Value

[double](#)

Examples

```
var coord = new Coordinate(95, 0);  
Console.WriteLine(coord.Latitude); // Output: 85
```

LatitudeR

Returns Latitude in Radians.

```
public double LatitudeR { get; }
```

Property Value

[double](#)

Examples

```
var coord = new Coordinate(90, 0);  
Console.WriteLine(coord.LatitudeR); // Output: 1.5707963267948966 (PI/2)
```

Longitude

Gets or sets the longitude in degrees. The value is wrapped to the range -180 to 180.

```
public double Longitude { get; set; }
```

Property Value

[double](#)[↗]

Examples

```
var coord = new Coordinate(0, 185);  
Console.WriteLine(coord.Longitude); // Output: 182.5
```

LongitudeR

Returns Longitude in Radians.

```
public double LongitudeR { get; }
```

Property Value

[double](#)[↗]

Examples

```
var coord = new Coordinate(0, 180);  
Console.WriteLine(coord.LongitudeR); // Output: 3.141592653589793 (PI)
```

Methods

Equals(object?)

Determines whether the specified [object](#)[↗] is equal to the current [Coordinate](#) object.

```
public override bool Equals(object? obj)
```

Parameters

obj [object](#)

The [object](#) to compare with the current object.

Returns

[bool](#)

true if the specified [object](#) is equal to the current object; otherwise, **false**.

GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

Returns

[int](#)

A hash code for the current object.

Operators

operator ==(Coordinate?, Coordinate?)

Compares two [Coordinate](#) objects for equality.

```
public static bool operator ==(Coordinate? left, Coordinate? right)
```

Parameters

left [Coordinate](#)

The first [Coordinate](#) to compare.

right [Coordinate](#)

The second [Coordinate](#) to compare.

Returns

[bool](#)

`true` if the coordinates are equal; otherwise, `false`.

operator !=(Coordinate, Coordinate)

Compares two [Coordinate](#) objects for inequality.

```
public static bool operator !=(Coordinate left, Coordinate right)
```

Parameters

`left` [Coordinate](#)

The first [Coordinate](#) to compare.

`right` [Coordinate](#)

The second [Coordinate](#) to compare.

Returns

[bool](#)

`true` if the coordinates are not equal; otherwise, `false`.

Class Dms

Namespace: [Deodesy.Library](#)

Assembly: Deodesy.Library.dll

Provides methods for handling Degrees, Minutes, Seconds (DMS) conversions and formatting.

```
public static class Dms
```

Inheritance

[object](#)  ← Dms

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Properties

DmsSeparator

Gets or sets the character used to separate degrees, minutes, and seconds in DMS strings. Defaults to a narrow no-break space (U+202F).

```
public static char DmsSeparator { get; set; }
```

Property Value

[char](#) 

Methods

CompassPoint(double, int)

Returns the compass point (to a given precision) for a supplied bearing.

```
public static string CompassPoint(double bearing, int precision = 3)
```

Parameters

bearing [double](#)

The bearing in degrees (0-360).

precision [int](#)

The precision of the compass point (1 for 8 points, 2 for 16 points, 3 for 32 points). Defaults to 3.

Returns

[string](#)

A string representing the compass point (e.g., "N", "NE", "NNE").

Examples

```
Console.WriteLine(Dms.CompassPoint(0));    // Output: N
Console.WriteLine(Dms.CompassPoint(22.5)); // Output: NNE
Console.WriteLine(Dms.CompassPoint(90));   // Output: E
Console.WriteLine(Dms.CompassPoint(315, 1)); // Output: N
```

Exceptions

[ArgumentOutOfRangeException](#)

Thrown if precision is not between 1 and 3.

Parse(string)

Parses a string representing degrees/minutes/seconds into numeric degrees.

```
public static double Parse(string dms)
```

Parameters

dms [string](#)

The DMS string to parse (e.g., "-3.62", "3 37 12W", "3°37'12"W").

Returns

[double](#) 

The parsed value in decimal degrees, or [NaN](#)  if parsing fails.

Examples

```
Console.WriteLine(Dms.Parse("53°08'45"N")); // Output: 53.14583333333336
Console.WriteLine(Dms.Parse("3 37 12W"));   // Output: -3.62
Console.WriteLine(Dms.Parse("-120.5"));      // Output: -120.5
```

Remarks

This method is very flexible on formats, allowing signed decimal degrees, or deg-min-sec optionally suffixed by compass direction (NSEW); a variety of separators are accepted.

Exceptions

[ArgumentNullException](#) 

Thrown if the input string is null or empty.

ToBearing(double, string, int)

Converts numeric degrees to deg/min/sec as a bearing (0°..360°).

```
public static string ToBearing(double deg, string format = "d", int dp = 2)
```

Parameters

deg [double](#) 

The decimal degrees bearing value.

format [string](#) 

The format to use: "d", "dm", or "dms".

dp [int](#) 

The number of decimal places.

Returns

[string](#)

A string representation of the bearing in DMS format.

Examples

```
Console.WriteLine(Dms.ToBearing(270.5, "dms", 0)); // Output: 270° 030' 000"
Console.WriteLine(Dms.ToBearing(-45, "d"));        // Output: 337.5°
```

ToCoordinate(string, string)

Returns a [Coordinate](#) object by parsing strings of latitude and longitude in DMS format.

```
public static Coordinate ToCoordinate(string latDms, string lonDms)
```

Parameters

latDms [string](#)

The latitude string in DMS format (e.g., "53°08'45"N").

lonDms [string](#)

The longitude string in DMS format (e.g., "001°50'00"W").

Returns

[Coordinate](#)

A [Coordinate](#) object representing the parsed latitude and longitude.

Examples

```
var coord = Dms.ToCoordinate("53°08'45"N", "001°50'00"W");
Console.WriteLine($"Latitude: {coord.Latitude}, Longitude: {coord.Longitude}");
// Output: Latitude: 53.14583333333336, Longitude: -1.8333333333333333
```

Exceptions

[ArgumentException](#)

Thrown if either latitude or longitude cannot be parsed.

ToDms(double, string, int)

Converts decimal degrees to deg/min/sec format.

```
public static string ToDms(double deg, string format = "d", int dp = 2)
```

Parameters

deg [double](#)

The decimal degrees value.

format [string](#)

The format to use: "d" (degrees), "dm" (degrees and minutes), or "dms" (degrees, minutes, and seconds).

dp [int](#)

The number of decimal places for seconds (or minutes if format is "dm", or degrees if format is "d").

Returns

[string](#)

A string representation of the degrees in the specified DMS format.

Examples

```
Console.WriteLine(Dms.ToDms(53.145833, "dms", 0)); // Output: 053° 008´ 045"
Console.WriteLine(Dms.ToDms(53.145833, "dm", 2)); // Output: 053° 008.75'
Console.WriteLine(Dms.ToDms(53.145833, "d", 4)); // Output: 053.1458°
```

Remarks

Degree, prime, double-prime symbols are added, but sign is discarded, though no compass direction is added. Degrees are zero-padded to 3 digits.

ToLat(double, string, int)

Converts numeric degrees to deg/min/sec latitude format (2-digit degrees, suffixed with N/S).

```
public static string ToLat(double deg, string format = "d", int dp = 2)
```

Parameters

deg [double](#)

The decimal degrees latitude value.

format [string](#)

The format to use: "d", "dm", or "dms".

dp [int](#)

The number of decimal places.

Returns

[string](#)

A string representation of the latitude in DMS format with N/S suffix.

Examples

```
Console.WriteLine(Dms.ToLat(53.145833, "dms", 0)); // Output: 053° 008' 045" N  
Console.WriteLine(Dms.ToLat(-25.1234, "dm", 2)); // Output: 025° 007.40' S
```

ToLon(double, string, int)

Converts numeric degrees to deg/min/sec longitude format (3-digit degrees, suffixed with E/W).

```
public static string ToLon(double deg, string format = "d", int dp = 2)
```

Parameters

deg [double](#)

The decimal degrees longitude value.

format [string](#)

The format to use: "d", "dm", or "dms".

dp [int](#)

The number of decimal places.

Returns

[string](#)

A string representation of the longitude in DMS format with E/W suffix.

Examples

```
Console.WriteLine(Dms.ToLon(-1.833333, "dms", 0)); // Output: 001° 050' 000" W
Console.WriteLine(Dms.ToLon(150.7654, "dm", 2)); // Output: 150° 045.92' E
```

Wrap180(double)

Constrains degrees to the range -180..+180 (for longitude) using a general sawtooth wave function.

```
public static double Wrap180(double degrees)
```

Parameters

degrees [double](#)

The degrees value to wrap.

Returns

[double](#)

The wrapped degrees value within the range -180 to 180.

Examples

```
Console.WriteLine(Dms.Wrap180(181)); // Output: 180.5
Console.WriteLine(Dms.Wrap180(-181)); // Output: -0.5
Console.WriteLine(Dms.Wrap180(360)); // Output: -90
```

Wrap360(double)

Constrains degrees to the range 0..360 (for bearings) using a general sawtooth wave function.

```
public static double Wrap360(double degrees)
```

Parameters

degrees [double](#)

The degrees value to wrap.

Returns

[double](#)

The wrapped degrees value within the range 0 to 360.

Examples

```
Console.WriteLine(Dms.Wrap360(361)); // Output: 180.5
Console.WriteLine(Dms.Wrap360(-1)); // Output: 359.5
Console.WriteLine(Dms.Wrap360(0)); // Output: 0
```

Wrap90(double)

Constrains degrees to the range -90..+90 (for latitude) using a general sawtooth wave function.

```
public static double Wrap90(double degrees)
```

Parameters

degrees [double](#)

The degrees value to wrap.

Returns

[double](#) 

The wrapped degrees value within the range -90 to 90.

Examples

```
Console.WriteLine(Dms.Wrap90(91)); // Output: 89  
Console.WriteLine(Dms.Wrap90(-91)); // Output: -89  
Console.WriteLine(Dms.Wrap90(180)); // Output: 0
```

Class LatLonSpherical

Namespace: [Deodesy.Library](#)

Assembly: Deodesy.Library.dll

Provides functions for spherical geodesy, based on the Haversine formula and other spherical geometry calculations. These functions are adapted from Chris Veness's original JavaScript geodesy library.

```
public class LatLonSpherical
```

Inheritance

[object](#)  ← LatLonSpherical

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

AlongTrackDistanceTo(Coordinate, Coordinate, Coordinate)

Calculates the along-track distance from a start point to the point on a great-circle path that is closest to a current point.

```
public double AlongTrackDistanceTo(Coordinate currentPoint, Coordinate startPoint,  
    Coordinate endPoint)
```

Parameters

currentPoint [Coordinate](#)

The current coordinate.

startPoint [Coordinate](#)

The starting coordinate of the great-circle path.

endPoint [Coordinate](#)

The ending coordinate of the great-circle path.

Returns

[double](#) 

The along-track distance in kilometers.

Examples

```
var start = new Coordinate(53.2611, -0.7972); // N of Lincoln
var end = new Coordinate(48.5734, 7.7161); // Strasbourg
var current = new Coordinate(53.1887, 0.1334); // E of Lincoln
var geo = new LatLonSpherical();
double alongTrackDist = geo.AlongTrackDistanceTo(current, start, end);
Console.WriteLine($"Along-track distance: {alongTrackDist:F2} km"); // Output: Along-track
distance: 53.7 km
```

Remarks

This is the distance from the start point to where a perpendicular from the current point crosses the great-circle path.

Exceptions

[ArgumentNullException](#) 

Thrown if `currentPoint`, `startPoint`, or `endPoint` is null.

Bearing(Coordinate, Coordinate)

Calculates the initial bearing (direction) from a start point to an end point.

```
public double Bearing(Coordinate startPoint, Coordinate endPoint)
```

Parameters

`startPoint` [Coordinate](#)

The starting coordinate.

endPoint [Coordinate](#)

The ending coordinate.

Returns

[double](#) 

The initial bearing in degrees, in the range 0 to 360.

Examples

```
var p1 = new Coordinate(51.51, 0.0); // London
var p2 = new Coordinate(38.89, -77.03); // Washington D.C.
var geo = new LatLonSpherical();
double bearing = geo.Bearing(p1, p2);
Console.WriteLine($"Initial Bearing: {bearing:F2}°"); // Output: Initial Bearing: 324.26°
```

Exceptions

[ArgumentNullException](#) 

Thrown if **startPoint** or **endPoint** is null.

CrossTrackDistance(Coordinate, Coordinate, Coordinate)

Calculates the cross-track distance from a current point to a great-circle path defined by two other points.

```
public double CrossTrackDistance(Coordinate currentPoint, Coordinate startPoint,
    Coordinate endPoint)
```

Parameters

currentPoint [Coordinate](#)

The current coordinate.

startPoint [Coordinate](#)

The starting coordinate of the great-circle path.

endPoint [Coordinate](#)

The ending coordinate of the great-circle path.

Returns

[double](#) 

The cross-track distance in kilometers.

Examples

```
var start = new Coordinate(53.2611, -0.7972); // N of Lincoln
var end = new Coordinate(48.5734, 7.7161); // Strasbourg
var current = new Coordinate(53.1887, 0.1334); // E of Lincoln
var geo = new LatLonSpherical();
double crossTrackDist = geo.CrossTrackDistance(current, start, end);
Console.WriteLine($"Cross-track distance: {crossTrackDist:F2} km"); // Output: Cross-track
distance: -31.93 km
```

Remarks

The cross-track distance is the shortest distance from the current point to the great-circle path.

Exceptions

[ArgumentNullException](#) 

Thrown if **currentPoint**, **startPoint**, or **endPoint** is null.

CrossingParallels(Coordinate, Coordinate, double)

Calculates the pair of meridians at which a great circle defined by two points crosses a given latitude.

```
public double[]? CrossingParallels(Coordinate startPoint, Coordinate endPoint,
double latitude)
```

Parameters

startPoint [Coordinate](#)

The starting coordinate of the great circle.

endPoint [Coordinate](#)

The ending coordinate of the great circle.

latitude [double](#)

The latitude (in degrees) for which to find the crossing meridians.

Returns

[double](#)[]

An array of two doubles representing the longitudes (in degrees, wrapped to -180 to 180) of the crossing points, or null if the great circle does not reach the given latitude.

Examples

```
var p1 = new Coordinate(0, 0); // Equator, Prime Meridian
var p2 = new Coordinate(30, 90); // 30°N, 90°E
var geo = new LatLonSpherical();
double[]? crossingLons = geo.CrossingParallels(p1, p2, 15); // Find where it crosses 15°N
if (crossingLons != null)
{
    Console.WriteLine($"Crossing Longitudes at 15°N: {crossingLons[0]:F2}°,
{crossingLons[1]:F2}°");
    // Output: Crossing Longitudes at 15°N: 27.65°, 152.35°
}
else
{
    Console.WriteLine("Great circle does not cross 15°N.");
}
```

Exceptions

[ArgumentNullException](#)

Thrown if **startPoint** or **endPoint** is null.

Destination(Coordinate, double, double)

Calculates the destination point and final bearing when traveling along a great circle arc for a given start point, initial bearing, and distance.

```
public Coordinate Destination(Coordinate startPoint, double distance, double bearing)
```

Parameters

startPoint [Coordinate](#)

The starting coordinate.

distance [double](#)

The distance to travel in kilometers.

bearing [double](#)

The initial bearing in degrees.

Returns

[Coordinate](#)

A [Coordinate](#) object representing the destination point.

Examples

```
var p1 = new Coordinate(51.51, 0.0); // London
var geo = new LatLonSpherical();
var destPoint = geo.destination(p1, 7794.0, 300.0); // Travel 7794 km on bearing 300°
Console.WriteLine($"Destination: Lat {destPoint.Latitude:F2},
Lon {destPoint.Longitude:F2}");
// Output: Destination: Lat 34.01, Lon -100.83
```

Exceptions

[ArgumentNullException](#)

Thrown if **startPoint** is null.

Distance(Coordinate, Coordinate)

Calculates the distance (in kilometers) between two points on the Earth's surface using the Haversine formula.

```
public double Distance(Coordinate startPoint, Coordinate endPoint)
```

Parameters

startPoint [Coordinate](#)

The starting coordinate.

endPoint [Coordinate](#)

The ending coordinate.

Returns

[double](#)[↗]

The distance in kilometers.

Examples

```
var p1 = new Coordinate(51.51, 0.0); // London
var p2 = new Coordinate(38.89, -77.03); // Washington D.C.
var geo = new LatLonSpherical();
double distance = geo.Distance(p1, p2);
Console.WriteLine($"Distance: {distance:F2} km"); // Output: Distance: 5908.38 km (approx)
```

Exceptions

[ArgumentNullException](#)[↗]

Thrown if **startPoint** or **endPoint** is null.

DistanceInNm(Coordinate, Coordinate)

Calculates the distance (in nautical miles) between two points on the Earth's surface using the Haversine formula.

```
public double DistanceInNm(Coordinate startPoint, Coordinate endPoint)
```

Parameters

startPoint [Coordinate](#)

The starting coordinate.

endPoint [Coordinate](#)

The ending coordinate.

Returns

[double](#)[↗]

The distance in nautical miles.

Examples

```
var p1 = new Coordinate(51.51, 0.0); // London
var p2 = new Coordinate(38.89, -77.03); // Washington D.C.
var geo = new LatLonSpherical();
double distanceNm = geo.DistanceInNm(p1, p2);
Console.WriteLine($"Distance: {distanceNm:F2} nm"); // Output: Distance: 3190.27 nm
```

Exceptions

[ArgumentNullException](#)[↗]

Thrown if **startPoint** or **endPoint** is null.

FinalBearing(Coordinate, Coordinate)

Calculates the final bearing (direction) when arriving at an end point from a start point.

```
public double FinalBearing(Coordinate startPoint, Coordinate endPoint)
```

Parameters

startPoint [Coordinate](#)

The starting coordinate.

endPoint [Coordinate](#)

The ending coordinate.

Returns

[double](#)[↗]

The final bearing in degrees, in the range 0 to 360.

Examples

```
var p1 = new Coordinate(51.51, 0.0); // London
var p2 = new Coordinate(38.89, -77.03); // Washington D.C.
var geo = new LatLonSpherical();
double finalBearing = geo.FinalBearing(p1, p2);
Console.WriteLine($"Final Bearing: {finalBearing:F2}°"); // Output: Final Bearing: 229.31°
```

Remarks

The final bearing will differ from the initial bearing by varying degrees according to distance and latitude.

Exceptions

[ArgumentNullException](#)[↗]

Thrown if **startPoint** or **endPoint** is null.

Intermediate(Coordinate, Coordinate, double)

Calculates the point at a given fraction along the great-circle path between two points.

```
public Coordinate Intermediate(Coordinate startPoint, Coordinate endPoint, double fraction)
```

Parameters

startPoint [Coordinate](#)

The starting coordinate.

endPoint [Coordinate](#)

The ending coordinate.

fraction [double](#)[↗]

The fraction along the path (0.0 for startPoint, 1.0 for endPoint, 0.5 for midpoint).

Returns

[Coordinate](#)

A [Coordinate](#) object representing the intermediate point.

Examples

```
var p1 = new Coordinate(51.51, 0.0); // London
var p2 = new Coordinate(38.89, -77.03); // Washington D.C.
var geo = new LatLonSpherical();
var intermediatePoint = geo.Intermediate(p1, p2, 0.25); // 25% of the way
Console.WriteLine($"Intermediate Point (25%): Lat {intermediatePoint.Latitude:F2}, Lon {intermediatePoint.Longitude:F2}");
// Output: Intermediate Point (25%): Lat 53.82, Lon -21.65
```

Exceptions

[ArgumentNullException](#)[↗]

Thrown if **startPoint** or **endPoint** is null.

Intersection(Coordinate, double, Coordinate, double)

Calculates the point of intersection of two great-circle paths.

```
public Coordinate? Intersection(Coordinate firstPoint, double firstBearing, Coordinate
secondPoint, double secondBearing)
```

Parameters

firstPoint [Coordinate](#)

The starting point of the first path.

firstBearing [double](#) 

The initial bearing of the first path in degrees.

secondPoint [Coordinate](#)

The starting point of the second path.

secondBearing [double](#) 

The initial bearing of the second path in degrees.

Returns

[Coordinate](#)

A [Coordinate](#) object representing the intersection point, or null if no intersection or paths are coincident.

Examples

```
var p1 = new Coordinate(51.8853, 0.235); // Cambridge
var brng1 = 108.549; // Bearing from Cambridge
var p2 = new Coordinate(49.0034, 2.5735); // Paris
var brng2 = 32.435; // Bearing from Paris
var geo = new LatLonSpherical();
var intersection = geo.Intersection(p1, brng1, p2, brng2);
if (intersection != null)
{
    Console.WriteLine($"Intersection: Lat {intersection.Latitude:F2},
Lon {intersection.Longitude:F2}");
    // Output: Intersection: Lat 50.99, Lon 4.50 (near Brussels)
}
else
{
    Console.WriteLine("No intersection found.");
}
```

Exceptions

[ArgumentNullException](#)

Thrown if `firstPoint` or `secondPoint` is null.

MaxLatitude(Coordinate, double)

Calculates the maximum latitude reached when traveling on a great circle from a start point with a given initial bearing (using Clairaut's formula).

```
public double MaxLatitude(Coordinate startPoint, double bearing)
```

Parameters

`startPoint` [Coordinate](#)

The starting coordinate.

`bearing` [double](#) 

The initial bearing in degrees.

Returns

[double](#) 

The maximum latitude in degrees.

Examples

```
var p1 = new Coordinate(51.51, 0.0); // London
var geo = new LatLonSpherical();
double maxLat = geo.MaxLatitude(p1, 90.0); // Bearing East
Console.WriteLine($"Max Latitude: {maxLat:F2}°"); // Output: Max Latitude: 51.51°
```

Remarks

The maximum latitude is independent of longitude; it will be the same for all points on a given latitude. Negate the result for the minimum latitude (in the Southern Hemisphere).

Exceptions

[ArgumentNullException](#) 

Thrown if `startPoint` is null.

MidPoint(Coordinate, Coordinate)

Calculates the midpoint between two points on the Earth's surface.

```
public Coordinate MidPoint(Coordinate startPoint, Coordinate endPoint)
```

Parameters

`startPoint` [Coordinate](#)

The starting coordinate.

`endPoint` [Coordinate](#)

The ending coordinate.

Returns

[Coordinate](#)

A [Coordinate](#) object representing the midpoint.

Examples

```
var p1 = new Coordinate(51.51, 0.0); // London
var p2 = new Coordinate(38.89, -77.03); // Washington D.C.
var geo = new LatLonSpherical();
var midpoint = geo.MidPoint(p1, p2);
Console.WriteLine($"Midpoint: Lat {midpoint.Latitude:F2}, Lon {midpoint.Longitude:F2}");
// Output: Midpoint: Lat 52.04, Lon -43.58
```

Exceptions

[ArgumentNullException](#) 

Thrown if `startPoint` or `endPoint` is null.

Namespace Deodesy.Library.Helpers

Classes

[Extensions](#)

Provides extension methods for various numeric operations, particularly for geographical calculations.

Class Extensions

Namespace: [Deodesy.Library.Helpers](#)

Assembly: Deodesy.Library.dll

Provides extension methods for various numeric operations, particularly for geographical calculations.

```
public static class Extensions
```

Inheritance

[object](#)  ← Extensions

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Methods

ToDegrees(double)

Converts a radian value to degrees.

```
public static double ToDegrees(this double value)
```

Parameters

value [double](#) 

The radian value to convert.

Returns

[double](#) 

The converted value in degrees.

Examples

```
double radians = Math.PI / 2;  
Console.WriteLine(radians.ToDegrees()); // Output: 90
```

ToFixed(double, int)

Rounds a double value to a specified number of decimal places.

```
public static double ToFixed(this double value, int dp = 2)
```

Parameters

value [double](#)

The double value to round.

dp [int](#)

The number of decimal places to round to. Defaults to 2.

Returns

[double](#)

The rounded double value.

Examples

```
double num = 123.45678;  
Console.WriteLine(num.ToFixed(2)); // Output: 123.46
```

ToRadians(double)

Converts a degree value to radians.

```
public static double ToRadians(this double value)
```

Parameters

value [double](#)

The degree value to convert.

Returns

[double](#)

The converted value in radians.

Examples

```
double degrees = 90;  
Console.WriteLine(degrees.ToRadians()); // Output: 1.5707963267948966 (PI/2)
```

Wrap180(double)

Wraps a degree value to the range -180 to +180, typically used for longitude.

```
public static double Wrap180(this double value)
```

Parameters

value [double](#)

The degree value to wrap.

Returns

[double](#)

The wrapped degree value.

Examples

```
Console.WriteLine(181.0.Wrap180()); // Output: -179  
Console.WriteLine((-181.0).Wrap180()); // Output: 179
```

Wrap360(double)

Wraps a degree value to the range 0 to 360, typically used for bearings.

```
public static double Wrap360(this double value)
```

Parameters

value [double](#)

The degree value to wrap.

Returns

[double](#)

The wrapped degree value.

Examples

```
Console.WriteLine(361.0.Wrap360()); // Output: 1  
Console.WriteLine((-1.0).Wrap360()); // Output: 359
```

Wrap90(double)

Wraps a degree value to the range -90 to +90, typically used for latitude.

```
public static double Wrap90(this double value)
```

Parameters

value [double](#)

The degree value to wrap.

Returns

[double](#)

The wrapped degree value.

Examples

```
Console.WriteLine(91.0.Wrap90()); // Output: 89  
Console.WriteLine((-91.0).Wrap90()); // Output: -89
```